

SAT and Exact Counting Below Four Gates per Input

Samuel Schlesinger

September 7, 2026

Abstract

For every fixed $\varepsilon > 0$, a Boolean circuit with n inputs and s gates can be counted exactly in time $\text{poly}_\varepsilon(n + s + 1)2^{(1/4+\varepsilon)(s+1)}$. Gates compute arbitrary functions of at most two inputs, and the algorithm permits exponential space. It therefore beats exhaustive search when $s \leq (4 - \gamma)n$ for any fixed $0 < \gamma < 4$. The proof balances enumeration of the used inputs against contraction of a counting network whose cycle rank is at most the gate count minus the used-input count plus one. This is a quantitative application of established graph-width and contraction methods.

1 The guarantee and the algorithm

The algorithm replaces a processed region by a table of conditional counts on its shared boundary. Internal assignments with the same boundary values have identical effects on the remaining constraints, so only their multiplicities must be retained. A counting invariant justifies this compression; graph structure bounds the table's size. Input enumeration provides a complementary bound, since a larger used-input count leaves a smaller consistency-cycle budget.

Let C be an acyclic circuit over the full binary basis B_2 , with n inputs, s gates, and one output. Each gate computes an arbitrary Boolean function of at most two inputs; constants and wiring are free, and fan-out is unrestricted. Write $\#\text{SAT}(C) = |\{x \in \{0, 1\}^n : C(x) = 1\}|$. Polynomial prefactors may depend on the fixed positive parameter in their subscript. Arithmetic costs include the bit lengths of the integers being counted.

Theorem 1. *For every fixed $\varepsilon > 0$, exact counting and satisfiability of C can be computed deterministically in time*

$$\boxed{\text{poly}_\varepsilon(n + s + 1)2^{(1/4+\varepsilon)(s+1)}}. \quad (1)$$

Space is bounded by a polynomial times the same exponential factor.

The proof gives a sharper bound before eliminating the number of used inputs. Prune to the output cone, substitute constants, and remove irrelevant pins and identity gates, repeating pruning after each simplification until no rule applies. Let C' have b used inputs and $q \leq s$ gates. Then $\#\text{SAT}(C) = 2^{n-b} \#\text{SAT}(C')$. This polynomial-time normalization need not find a smallest circuit. Constant and designated-input outputs are counted directly; below assume $q \geq 1$.

Proposition 2. *For every fixed $\delta > 0$, the count can be computed in time*

$$\text{poly}_\delta(n + s + 1)2^{\min\{b, (1/3+\delta)(s+1-b)\}}, \quad (2)$$

with space bounded by a polynomial times the same exponential factor.

The algorithm has four steps:

1. Normalize C , obtaining a circuit C' on b used inputs.
2. Encode C' as integer tables on a graph of cycle rank $\kappa \leq s + 1 - b$. Reduce it exactly to a scalar factor and a simple cubic graph K with $N \leq 2\kappa$ vertices.
3. Construct a vertex order of K with cutwidth $w \leq (1/3 + \delta)\kappa + O_\delta(1)$.
4. If $b \leq w$, enumerate the inputs of C' ; otherwise evaluate its tables along the order, keeping a table on the crossing edges. Return $2^{n-b} \#SAT(C')$.

If K is empty, its scalar already gives $\#SAT(C')$. The next three sections prove the encoding, the width bound, and the time bound. Only the stated graph-width theorem is imported.

2 Circuit values give a small cycle budget

Conditional counts in a three-gate circuit. Let $a = x \vee y$, $c = y \oplus z$, and let the output be $a \wedge c$. For acceptance both a and c must be one. After summing over x and z separately, the two tables, indexed by the same y , are

y	$\#\{x : x \vee y = 1\}$	$\#\{z : y \oplus z = 1\}$	product
0	1	1	1
1	2	1	2

Thus the count is $1 \cdot 1 + 2 \cdot 1 = 3$. Multiplication combines choices that agree on y ; addition joins the disjoint cases for y . Summing the columns independently would give $(1 + 2)(1 + 1) = 6$: it would allow two uses of one input to disagree. The general boundary table prevents this same error. The entry two records the fact that either value of x makes $x \vee 1$ true.

Introduce one variable for each used input and each gate output. Each gate v imposes its equation $z_v = \text{op}_v(\text{predecessor values})$; add the unary constraint $z_{\text{out}} = 1$. Form the bipartite incidence multigraph H between these variables and constraints. Each occurrence is an edge, including repeated uses of one value at the same gate. If $\ell \leq 2q$ counts the gate input pins, then

$$|V(H)| = 2q + b + 1, \quad |E(H)| = q + \ell + 1.$$

Every remaining value leads to the output, so H is connected. Thus

$$\kappa := |E(H)| - |V(H)| + 1 = \ell - q - b + 1 \leq q + 1 - b \leq s + 1 - b. \quad (3)$$

Give every incidence edge an independent binary index. At each variable vertex, put the table that is one exactly when all its incident indices agree. A degree-one equality table is $(1, 1)$. At each constraint vertex, put its zero-one truth table. The *contraction* is the sum, over all edge-index assignments, of the product of all local entries. A high-degree equality table is represented by its equality rule, without listing its entries; it is expanded into ternary tables in the next proof.

Lemma 3. *This contraction equals $\#SAT(C')$. In polynomial time it reduces to a scalar factor times a network on an empty graph or a simple connected cubic graph K . All remaining tables have at most eight entries, and, if $N = |V(K)| > 0$,*

$$N = 2(\kappa(K) - 1) \leq 2(\kappa - 1). \quad (4)$$

Proof. For any assignment of the b inputs, topological evaluation uniquely determines all gate values. Equality tables uniquely determine their incidence indices. Hence an accepting input contributes exactly one to the sum, and every other input contributes zero.

Replace each equality vertex of degree $D \geq 4$ by a tree of $D - 2$ ternary equality vertices. For each external assignment, there is exactly one internal extension if all its bits agree and none otherwise. The replacement adds equally many vertices and edges, preserving cycle rank. Since the original variable degrees sum to $q + \ell + 1$, the resulting subcubic network has at most $4q + 2$ vertices and $O(q + 1)$ edges.

Now repeat the following exact operations:

1. Absorb a rank-zero table into a scalar accumulator.
2. Sum a loop over its two diagonal entries: $T'(a) = T(0, 0, a) + T(1, 1, a)$, where a denotes the other ports.
3. At a loop-free vertex of degree one or two, contract one edge into its neighbor, summing over that edge's bit. Any second shared edge becomes a loop.
4. If two cubic vertices share $j \in \{2, 3\}$ edges, contract all j together, summing over their 2^j assignments.

Each operation uses constantly many integer additions and multiplications, preserves the contraction, decreases $|V| + |E|$, and leaves every table with at most three ports. Ordinary edge contraction preserves cycle rank; eliminating a loop lowers it by one; contracting j parallel edges lowers it by $j - 1$. The nonempty remainder stays connected. When no operation applies, it is simple and cubic. Therefore $|E(K)| = 3N/2$ and $\kappa(K) = N/2 + 1$, proving (4). $\square$

These operations count internal assignments conditional on the surviving boundary. In particular, a loop is summed over two consistent values, and parallel edges are summed jointly. The edges carry constraints and cannot simply be deleted.

3 A narrow order for the cubic remainder

A path decomposition consists of bags that cover the vertices, contain both endpoints of each edge in some bag, and contain each vertex in a consecutive interval of bags. Its width is the largest bag size minus one. The cutwidth of a vertex order is the maximum number of edges between a prefix and its complement.

We use the following theorem of Fomin and Høie [3, Theorem 5 and Section 4]: for each fixed $\eta > 0$, every sufficiently large simple graph of maximum degree three has a path decomposition of width at most $(1/6 + \eta)N$, constructible in polynomial time. The threshold and polynomial may depend on η .

The next inequality is standard; see, for example, the proof of Bodlaender et al. [1, Lemma 16]. We give a direct constructive proof for the cubic case used here.

Lemma 4. *From a width- k path decomposition of a simple cubic graph, one can construct a vertex order of cutwidth at most $k + 2$ in polynomial time.*

Proof. Represent each vertex v by the interval $I_v = [\text{first}(v) - 1/4, \text{last}(v) + 1/4]$ of its bag positions. At most $k + 1$ such intervals cover any point. For each edge $e = uv$, choose a distinct rational timestamp t_e in the interior of $I_u \cap I_v$. Such timestamps can be obtained by distinct small rational

perturbations of a bag containing u, v , using polynomially many bits. Order the vertices by the median t_v of their three incident timestamps, breaking ties arbitrarily.

At a cut t between distinct vertex timestamps, consider a crossing edge $e = uv$ with $t_u < t < t_v$. Charge it to u if $t_e > t$, and to v otherwise. The charged vertex's interval contains t . It receives at most one charge, since a cubic vertex has only one incident timestamp above its median and only one below it. There are consequently at most $k + 1$ crossing edges.

If a cut separates vertices with tied medians t , at most two vertices tie: distinct edge timestamps can be the median only of their endpoints. The same charging rule at t handles every crossing edge except possibly the one timestamped t itself. That adds at most one edge, giving $k + 2$. $\square$

Use the cited theorem with $\eta = \delta/2$ and apply Lemma 4. By (4), for sufficiently large K the resulting order has

$$w \leq (1/6 + \delta/2)N + 2 \leq (1/3 + \delta)\kappa + 2.$$

Below the fixed size threshold, any order has bounded cutwidth. This proves $w \leq (1/3 + \delta)\kappa + O_\delta(1)$ for every K and gives the polynomial-time layout step of the algorithm.

4 Frontier counting and the running time

Order the cubic vertices as $v_1, \dots, v_N$, and let F_i be the edges leaving the first i vertices. For each assignment $\beta \in \{0, 1\}^{F_i}$, store $P_i(\beta)$: the sum of the products of the processed tables over all processed internal edge values, with the frontier fixed to β . Initialize $P_0(\emptyset) = 1$.

Only processed constraints contribute, so a positive entry need not extend to a solution. The invariant permits us to forget the identities of internal histories, while retaining their count for each possible boundary.

At vertex v_i , let J_i be the edges back to the processed prefix, R_i the edges forward, and $S_i = F_{i-1} \setminus J_i = F_i \setminus R_i$ the unchanged frontier. The update is

$$P_i(\beta) = \sum_{\gamma \in \{0, 1\}^{J_i}} P_{i-1}(\beta|_{S_i}, \gamma) T_{v_i}(\gamma, \beta|_{R_i}). \quad (5)$$

Distributivity preserves the invariant. The final value $P_N(\emptyset)$, multiplied by the scalar from the reduction, is $\#\text{SAT}(C')$.

For a fixed new boundary assignment, each choice of consumed bits γ identifies an old entry and a compatible local entry. Their product counts matching assignments in disjoint regions; the sum joins disjoint choices of γ . The untouched frontier S_i stays fixed, retaining every value still shared with the unprocessed region.

Write $j = |J_i|$. The update uses fewer than $2^{|F_i|+j+1}$ arithmetic operations. Cubicity gives $|F_i| = |F_{i-1}| + 3 - 2j$. Since both frontiers have size at most w ,

$$|F_i| + j \leq w + 1 : \quad \text{use the new frontier for } j \leq 1 \text{ and the old for } j \geq 2.$$

Thus contraction takes $O(N2^w)$ arithmetic operations and $O(2^w)$ stored entries, keeping only two successive frontiers.

Every intermediate entry counts assignments to a subset of the $O(q + 1)$ binary indices in the expanded zero-one network, conditional on a boundary assignment. Its bit length is therefore $O(q + 1)$. Arithmetic has polynomial bit cost, and multiplication by 2^{n-b} is a shift producing an answer of at most $n + 1$ bits. Consequently the contraction runs in time $\text{poly}(n + s + 1)2^w$.

Why the space is exponential. The table has one entry per assignment to the crossing wires: w binary wires give up to 2^w entries. Each entry has only polynomially many bits. For SAT alone, Boolean entries suffice, but the table still has up to 2^w of them. Storing the table allows each partial computation to be reused; recomputing entries instead may increase the time. This is a space upper bound for the present implementation, not a lower bound on the space needed to achieve the exponent. The enumeration branch uses polynomial space.

Proof of Proposition 2. Lemmas 3 and 4 give polynomial-time preprocessing and $w \leq \alpha\kappa + c_\delta$, where $\alpha = 1/3 + \delta$ and c_δ is a constant. The algorithm chooses between contraction and enumeration using the computed integer w . Its time is at most

$$\text{poly}_\delta(n + s + 1)2^{\min\{b, w\}} \leq \text{poly}_\delta(n + s + 1)2^{\min\{b, \alpha\kappa\}}.$$

Here the constant factor 2^{c_δ} is absorbed into the polynomial prefactor. Equation (3) gives (2). The space bound follows from the two stored frontiers; enumeration itself uses polynomial space. $\square$

5 The four-gate threshold and the comparison

Proof of Theorem 1. Write $S = s + 1$ and $\alpha = 1/3 + \delta$. For $0 \leq b \leq S$,

$$\min\{b, \alpha(S - b)\} \leq \frac{\alpha}{1 + \alpha}S, \quad \frac{\alpha}{1 + \alpha} = \frac{1}{4} + \frac{9\delta}{16 + 12\delta}.$$

Proposition 2 with $\delta = \varepsilon$ proves (1), with the asserted space bound. SAT is decided by testing whether the count is positive. $\square$

Ignoring the fixed positive slack, the competing exponents are b and $(s + 1 - b)/3$. Their worst balance occurs at $s + 1 = 4b$: either there are few enough used inputs to enumerate, or the cycle budget left over after accounting for those inputs is small enough to contract. This is the source of the constant four.

Corollary 5. For each fixed $0 < \gamma < 4$, circuits with $s \leq (4 - \gamma)n$ admit deterministic SAT and exact counting in time $\text{poly}_\gamma(n + 1)2^{(1 - \gamma/8)n}$.

Proof. Set $\varepsilon = \gamma/[8(4 - \gamma)]$ in (1). Then $(1/4 + \varepsilon)(s + 1) \leq (1 - \gamma/8)n + (1/4 + \varepsilon)$. $\square$

Comparison with the $2.99n$ counting algorithm. Golovnev, Kulikov, Smal, and Tamaki give a gate-elimination algorithm for exact counting in the same unrestricted-fan-out B_2 model [4]. Their corrected preprint, Section 5.2, Remark 2, gives a running time $\text{poly}_\zeta(n)2^{(1 - c_\zeta)n}$ for circuits of size at most $(3 - \zeta)n$, where $c_\zeta > 0$ for each fixed $0 < \zeta < 3$. Thus $2.99n$ is an example within their range. Their decreasing measure bounds each circuit's size by $O_\zeta(s + n)$, and each branch eliminates an input. A depth-first evaluation therefore uses polynomial space.

Here the fixed-gap range reaches $(4 - \gamma)n$, using exponential space on the contraction branch. For example, at $s \leq 2.99n$, Theorem 1 gives time $\text{poly}_\rho(n + 1)2^{(0.7475 + \rho)n}$ for every fixed $\rho > 0$. This is a different time–space guarantee; it does not supply a polynomial-space extension of their result to the four-gate range.

Scope and antecedents. The positive slack is fixed before the input size grows; a uniform endpoint bound $\text{poly}(n + s)2^{s/4}$ and a polynomial-space algorithm with this exponent are not established here. Width-based tensor contraction and counting representations are established methods [2, 5]; the note spells out their circuit-specific cycle budget and exponent calculation. Historical priority of the precise bound and its best-known status among algorithms permitting exponential space remain unestablished. The accompanying repository checks finite components of the proof; its `rs/` directory also implements the complete automatic algorithm. The source-to-proof map and fixed-parameter resource argument are in `rs/ALGORITHM.md`. Its retained structured timings illustrate small-kernel behavior, rather than establishing the worst-case guarantee empirically.

References

- [1] Hans L. Bodlaender, Edouard Bonnet, Lars Jaffke, Dušan Knop, Paloma T. Lima, Martin Milanič, Sebastian Ordyniak, Sukanya Pandey, and Ondřej Suchý. Treewidth is NP-complete on cubic graphs. *Electronic Journal of Combinatorics*, 32(3):P3.36, 2025. doi: 10.37236/13205. URL <https://www.combinatorics.org/ojs/index.php/eljc/article/view/v32i3p36>.
- [2] Jeffrey M. Dudek, Leonardo Dueñas-Osorio, and Moshe Y. Vardi. Efficient contraction of large tensor networks for weighted model counting through graph decompositions. arXiv:1908.04381v2, 2020. URL <https://arxiv.org/abs/1908.04381v2>.
- [3] Fedor V. Fomin and Kjartan Høie. Pathwidth of cubic graphs and exact algorithms. *Information Processing Letters*, 97(5):191–196, 2006. doi: 10.1016/j.ipl.2005.10.012. URL <https://fedorvf.github.io/articles/2006/2006b.pdf>.
- [4] Alexander Golovnev, Alexander S. Kulikov, Alexander V. Smal, and Suguru Tamaki. Gate elimination: Circuit size lower bounds and #SAT upper bounds. *Theoretical Computer Science*, 719:46–63, 2018. doi: 10.1016/j.tcs.2017.11.008. URL <https://eccc.weizmann.ac.il/report/2016/022/revision/2/download>. Corrected preprint: ECCC TR16-022, revision 2, 2017.
- [5] Igor L. Markov and Yaoyun Shi. Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3):963–981, 2008. doi: 10.1137/050644756. URL <https://arxiv.org/abs/quant-ph/0511069>.